

How Far Can You Get Without a GPU? A Systematic Benchmark of Lightweight Hallucination Detection Across Question Answering, Dialogue, and Summarisation

Kriti Faujdar

Independent Researcher
kritifaujdar@gmail.com

Smit Kadvani

Independent Researcher
smit.kadvani@gmail.com

Abstract

Hallucination detection has become a pressing requirement for trustworthy AI deployment at scale. The most accurate detection methods depend on GPU-intensive inference, proprietary API calls, or white-box access to the generating model. This puts them out of reach for resource-constrained researchers and practitioners. In this paper, we explore a practical alternative: how well can hallucination detection perform using only lightweight, CPU-feasible methods built on publicly available models? We systematically benchmark four such detectors, ROUGE-L, semantic similarity, BERTScore, and a Natural Language Inference (NLI) detector based on a FEVER-trained DeBERTa model, together with a score-level ensemble of similarity and NLI. We evaluate them across all three tasks of the HaluEval benchmark: question answering (QA), dialogue, and summarisation. We calibrate each method on a held-out validation split and evaluate it on 2,000 test instances per task, and we report bootstrap confidence intervals for the main results. We find that the similarity-NLI ensemble is the most consistent method, but that absolute performance is highly task-dependent. The ensemble ranks best on QA ($F1 = 0.792$, $AUC-ROC = 0.873$) and on dialogue ($F1 = 0.694$, $AUC-ROC = 0.749$), where NLI is the strongest standalone method; on summarisation every method performs near chance ($AUC-ROC$ between 0.469 and 0.574). We then ask whether that failure is intrinsic to lightweight detection or an artifact of our single-pass design, and find it is largely the latter. Raising the premise budget from 800 to 1600 characters lifts summarisation $AUC-ROC$ from 0.567 to 0.629, and replacing single-pass scoring with sentence-level chunk aggregation reaches 0.683, still on CPU and with the same model, though at roughly twenty times the NLI inference. Summarisation remains by far the hardest task, but our results do not support treating lightweight detection as intrinsically unsuited to it. All ex-

periments run on a standard laptop CPU using public models.

1 Introduction

Large language models (LLMs) produce fluent text across question answering, dialogue, summarisation, and many other natural language processing tasks (Brown et al., 2020; Touvron et al., 2023). They also hallucinate. A hallucination is text that reads as fluent and plausible but is factually incorrect or unsupported by any available source (Ji et al., 2023). In high-stakes domains such as healthcare, law, and education, hallucinated outputs pose serious risks. This risk drives active research into automatic hallucination detection (Maynez et al., 2020).

The most accurate hallucination detection methods all depend on resources that many researchers, educators, and practitioners do not have: GPUs, proprietary APIs, or model internals. We discuss these methods in detail in Section 2.

We take a constrained position. Instead of proposing another resource-intensive detector, we ask a question. How far can hallucination detection progress using only lightweight, CPU-feasible methods that rely on publicly available models, make no API calls, and assume no access to the generating model? This question matters for two reasons. First, it sets a realistic performance baseline for the large community that works without specialised hardware. Second, it shows where accessible methods succeed and where they break down. Papers that compare a single expensive method against weak baselines hide this information.

We benchmark four CPU-feasible detectors that span the lexical, embedding, and inference paradigms, ROUGE-L, semantic similarity, BERTScore, and an NLI-based detector, together with a score-level ensemble of similarity and NLI. We evaluate them across all three tasks of the

HaluEval benchmark (Li et al., 2023): question answering, dialogue, and summarisation. Our contributions are:

- A systematic benchmark of four CPU-feasible hallucination detectors and a score-level ensemble across three tasks. We calibrate thresholds on held-out validation data and evaluate on 2,000 test instances per task.
- The finding that the similarity-NLI ensemble is the most consistent method, ranking best on both QA and dialogue, while absolute performance remains strongly task-dependent. NLI is the strongest standalone method on dialogue.
- A diagnosis of the summarisation failure. Every method performs near chance there, but two controlled follow-ups show this is driven largely by single-pass truncation: enlarging the premise budget and switching to sentence-level chunk aggregation both recover substantial performance without extra hardware, the latter at a substantial increase in CPU inference.
- Bootstrap confidence intervals for the main results and paired bootstrap comparisons of AUC-ROC between the leading methods on each task, establishing which rankings our data resolve and which it does not.
- An error analysis of the NLI detector’s failure cases, and practical guidance for method selection under computational constraints.

2 Related Work

Ji et al. (2023) survey hallucination in natural language generation. They distinguish faithfulness (relation to a provided source) from factuality (relation to world knowledge). HaluEval annotates candidates against a provided source, so our study falls in the faithfulness category. Detection methods group by resource requirement. *Model-as-judge* methods prompt a strong LLM to evaluate faithfulness. They reach high human agreement but need frontier-model access and add per-query costs (Liu et al., 2023). *Sampling-based* methods such as SelfCheckGPT (Manakul et al., 2023) measure consistency across stochastic samples. They run in a black-box setting but need several times the generation cost. *Internal-state* methods analyse token

probabilities or activations (Farquhar et al., 2024). They need white-box access, which API-only models do not allow. *Retrieval and fact-verification* pipelines decompose candidates into atomic claims and verify each against retrieved evidence (Min et al., 2023). This adds operational complexity.

The methods we evaluate belong to two lighter families. *NLI-based* methods treat the source as premise and the candidate as hypothesis. Falke et al. (2019) first repurposed NLI for summary faithfulness and found that standard NLI corpora transferred poorly. Later work trained on the FEVER fact-verification dataset (Thorne et al., 2018). Laurer et al. (2024) released DeBERTa models fine-tuned on MultiNLI, FEVER, and Adversarial NLI, which we adopt. Laban et al. (2022) introduced SummaC and showed that NLI methods beat lexical metrics for inconsistency detection. *Overlap-based* metrics include ROUGE (Lin, 2004), BERTScore (Zhang et al., 2020), and Sentence-BERT similarity (Reimers and Gurevych, 2019). These are cheap to compute but can be misled when hallucinations stay topically coherent with the source.

HaluEval (Li et al., 2023) provides 10,000 samples for each of three tasks: QA, dialogue, and summarisation. To build hallucinated candidates, the authors prompt ChatGPT for plausible but incorrect content. Prior work has mostly reported QA results alone. We provide a systematic comparison of lightweight methods across all three tasks under a common calibration protocol.

3 Dataset

We use all three HaluEval tasks (Li et al., 2023) from HuggingFace (pminervini/HaluEval). Each sample provides a source, a task-specific context, and a candidate labelled faithful or hallucinated (Table 1). For each task we draw 3,000 of the 10,000 samples with seed 42, a size chosen to keep NLI inference practical on a laptop CPU while leaving a test set large enough for stable estimates (bootstrap intervals of ± 0.015 to ± 0.026 AUC-ROC; Appendix B). We split them into a 1,000-instance validation set for threshold and ensemble-weight calibration, and a 2,000-instance test set for all reported metrics. Stratified sampling keeps the class distribution close to balanced. This split keeps every thresholding decision off the test data.

Table 1: Structure of the three HaluEval tasks.

Task	Source	Context	Candidate
QA	Knowledge passage	Question	Answer
Dial.	Conversation knowl.	History	Response
Summ.	Document	—	Summary

4 Methodology

Problem formulation. Given a source S , optional context C , and candidate C' , we predict $y \in \{0, 1\}$, where $y = 1$ denotes hallucination. Each method produces a score s . A threshold τ turns the score into a binary prediction. For every method, higher s means a higher chance of hallucination; the score definitions below make this explicit.

The four detectors and the ensemble. **ROUGE-L:** s is the longest-common-subsequence F-measure between source and candidate. **Semantic Similarity:** s is the cosine similarity between all-MiniLM-L6-v2 (Reimers and Gurevych, 2019) embeddings of source and candidate (≈ 22 M parameters). **BERTScore:** s is the token-level contextual F-measure with a DistilBERT backbone (Zhang et al., 2020). For these three overlap detectors we use the raw score, *not* one minus it: HaluEval’s hallucinated candidates are generated to be topically plausible, so they tend to overlap *more* with the source than faithful ones do, and on QA and dialogue higher overlap does indicate hallucination. **NLI:** the checkpoint MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli, DeBERTa-v3-base fine-tuned on MultiNLI, FEVER, and ANLI (Laurer et al., 2024) (≈ 184 M parameters). The premise is the source (truncated to 800 characters) plus context, the hypothesis is the candidate, and $s = 1 - P(\text{entailment})$. **Ensemble:** $s = \alpha s_{\text{NLI}} + (1 - \alpha) s_{\text{sim}}$, with α chosen by grid search over $\{0.1, \dots, 0.9\}$ to maximise validation AUC-ROC. The selected weights are $\alpha = 0.4$ (QA), $\alpha = 0.9$ (dialogue), and $\alpha = 0.3$ (summarisation). These weights already show that dialogue relies almost entirely on the NLI signal, while QA benefits from a more balanced mix. Orientation is fixed in advance for every detector; the calibrated threshold sets only the operating point and cannot reverse the direction of a score.

Input length limits. Three of the four detectors see a truncated source. The NLI premise is capped at 800 characters, BERTScore references at 512

characters, and all-MiniLM-L6-v2 truncates its inputs at its default maximum of 256 tokens; only ROUGE-L uses the full source. These limits are fixed implementation choices, not tuned values. Section 6 examines their effect on summarisation.

Calibration and metrics. For each method, we select τ on the validation split by maximising the Youden index (TPR – FPR) and apply it unchanged to the test split. We report Accuracy, Precision, Recall, F1 (positive = hallucinated), and AUC-ROC, computed with scikit-learn (Pedregosa et al., 2011). Because point estimates cannot show whether a difference between two methods is meaningful, we compute 95% bootstrap confidence intervals for AUC-ROC on the main results, resampling the test set with replacement over 2,000 replicates, and we compare the best and second-best method on each task by bootstrapping their paired AUC-ROC difference on the same replicates. All experiments run on a standard laptop CPU. We batch embeddings at 32 and NLI at 16. Our code is available at <https://github.com/fkriti/hallucination-detection-nli>.

5 Results

Table 2 reports all methods across all tasks. Figure 1 shows the task-dependent pattern.

Question answering. QA is the easiest task. The ensemble is strongest on accuracy (.803), recall (.766), F1 (.792), and AUC-ROC (.873). It beats the best standalone method by about 6 points on F1 and about 5 points on AUC-ROC. NLI reaches the highest precision (.865) but low recall (.625), a conservative operating point. BERTScore (.821 AUC) edges ROUGE-L (.817) and clearly beats semantic similarity (.736). Contextual token overlap is therefore a stronger signal than sentence-level cosine similarity. The ensemble’s gain shows that the similarity and NLI signals complement each other.

Dialogue. Dialogue is harder. The best AUC-ROC falls to .749 (ensemble) and .713 (NLI). The ensemble again ranks first on accuracy, precision, F1 and AUC-ROC, while NLI is the strongest standalone method and reaches the highest recall (.781). Entailment reasoning therefore transfers to the conversational setting, and combining it with similarity still helps. Lexical and embedding methods cluster at .61–.66 AUC-ROC, weaker than on QA because responses are shorter and depend more on context.

Table 2: Hallucination detection performance across the three HaluEval tasks (test $n = 2,000$ per task). Best result per task and metric in bold. On summarisation, the AUC-ROC gap between the two best methods is not statistically significant (Table 3), and BERTScore’s high recall and F1 there reflect labelling nearly every candidate as hallucinated.

Task	Method	Accuracy	Precision	Recall	F1	AUC-ROC
QA	ROUGE-L	0.753	0.794	0.670	0.726	0.817
	Semantic Similarity	0.655	0.628	0.730	0.675	0.736
	BERTScore	0.745	0.754	0.713	0.733	0.821
	NLI	0.768	0.865	0.625	0.725	0.795
	Ensemble (Sim+NLI)	0.803	0.821	0.766	0.792	0.873
Dialogue	ROUGE-L	0.593	0.577	0.640	0.607	0.612
	Semantic Similarity	0.609	0.636	0.475	0.544	0.656
	BERTScore	0.599	0.572	0.722	0.638	0.637
	NLI	0.637	0.600	0.781	0.679	0.713
	Ensemble (Sim+NLI)	0.672	0.640	0.756	0.694	0.749
Summarisation	ROUGE-L	0.545	0.531	0.610	0.568	0.563
	Semantic Similarity	0.502	0.495	0.855	0.627	0.528
	BERTScore	0.494	0.492	0.991	0.658	0.469
	NLI	0.553	0.533	0.712	0.609	0.567
	Ensemble (Sim+NLI)	0.552	0.549	0.490	0.518	0.574

Summarisation: a systematic failure mode. Every method performs near chance, with AUC-ROC from .469 (BERTScore) to .574 (ensemble). BERTScore’s value below .5 reflects our fixed orientation rather than a usable inverse signal: reversing its direction gives only .532. The high F1 for BERTScore (.658) and semantic similarity (.627) is misleading. It comes from extreme recall (.991, .855) with near-chance precision. These methods label almost everything hallucinated. Two factors plausibly contribute. First, a property of the data: HaluEval summarisation hallucinations are subtle factual edits inside long, otherwise faithful summaries of long documents, so the faithful remainder dominates the overlap metrics and they cannot localise the inconsistent span. Second, a property of our configuration: three of the four detectors see a truncated source, and the NLI detector’s 800-character premise in particular cannot reach the full document. Section 6 separates these two explanations experimentally and shows that the second carries much of the weight, so the figures in this row should be read as the behaviour of a single-pass configuration rather than as a ceiling on lightweight detection.

Synthesis. On QA and dialogue the ensemble ranks first on every metric except precision on QA and recall on dialogue, where NLI leads. These are rankings; only the AUC-ROC gap to the runner-up is significance-tested (Table 3). Inference-based signals, alone or combined with similarity, are therefore the most reliable choice when lightweight detection is viable. All methods show a steep difficulty gradient from QA (AUC up to .873) through dialogue (.749) to summarisation (.574). The viability of GPU-free detection depends strongly on the task. Figure 2 shows the per-task ROC curves. The gradient appears as a progressive collapse of all curves towards the diagonal: well separated on QA, compressed on dialogue, and indistinguishable from chance on summarisation.

Statistical reliability. Point estimates alone cannot show whether one method genuinely outperforms another, so we resample the test set with replacement (2,000 bootstrap replicates, seed 42) and recompute AUC-ROC. Full per-method intervals appear in Appendix B; they are narrow, typically ± 0.02 AUC-ROC. Table 3 bootstraps the paired difference between the best method and the runner-up on each task. The ensemble’s advan-



Figure 1: The four detectors and the ensemble across all three tasks. The task-dependent ranking and the degradation from QA to summarisation are visible in both metrics.

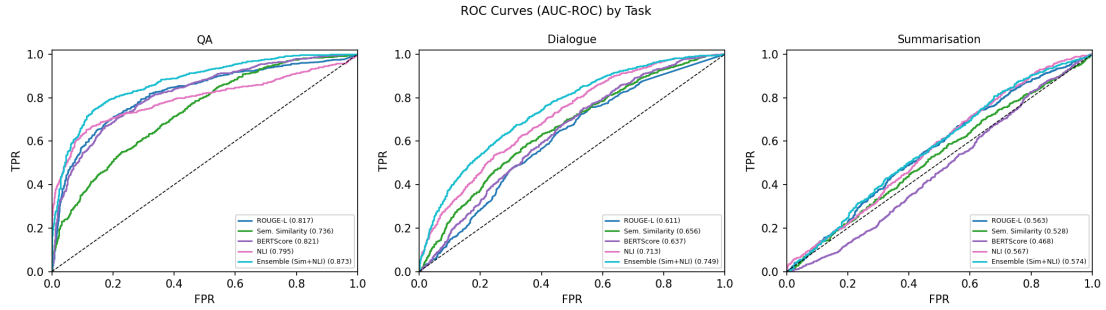


Figure 2: Receiver Operating Characteristic curves for the four detectors and the ensemble on each task. Separation from the diagonal (chance) degrades systematically from QA (left) to summarisation (right).

Table 3: Paired bootstrap of the AUC-ROC gap between the best and second-best method on each task. An interval excluding zero means the ranking is resolved.

Task	Best vs. runner-up	ΔAUC	95% CI
QA	Ensemble vs. BERTScore	0.052	[0.032, 0.073]
Dial.	Ensemble vs. NLI	0.036	[0.015, 0.057]
Summ.	Ensemble vs. NLI	0.007	[-0.009, 0.022]

tage is resolved on QA and dialogue, where the interval excludes zero. On summarisation the interval spans zero, so our data does *not* establish a ranking among the methods there. This is consistent with all of them sitting close to chance, and we therefore avoid claiming a best method for that task. BERTScore’s summarisation interval, [0.442, 0.493], lies entirely below 0.5, but this does not make it a useful inverse detector: orientation is fixed in advance, and reversing BERTScore’s direction gives an AUC-ROC of only 0.532. It is near chance either way.

Computational footprint. The four detectors and the ensemble run on a standard laptop CPU

with no GPU. Table 4 reports model size and measured throughput on an Apple M4 laptop. The lexical and embedding methods are computationally inexpensive. ROUGE-L processes roughly 900 candidates per second. The NLI detector, with its 184M-parameter model, is the bottleneck at 3.9 candidates per second. It scores a full 2,000-instance test split in about nine minutes. The ensemble adds no model beyond NLI and similarity, so NLI dominates its cost. Even the slowest setup stays practical for offline evaluation. The similarity-only and lexical methods are fast enough for interactive use.

6 Revisiting the Summarisation Failure

The result above is easy to over-read. Reporting that every lightweight method lands near chance on summarisation invites the conclusion that lightweight detection is intrinsically unsuited to the task. Before drawing that conclusion we need to rule out a simpler explanation: our own configuration. The single-pass detector truncates the source to 800 characters, while summarisation

Table 4: Model size and measured CPU throughput on an Apple M4 (10 cores, 16 GB RAM) with default PyTorch threading. Throughput is candidates scored per second.

Method	Params	Throughput
ROUGE-L	—	~900/s
Sem. Similarity	22M	~58/s
BERTScore	66M	~62/s
NLI	184M	~3.9/s
Ensemble	206M	~3.7/s

Table 5: Summarisation performance as the NLI premise budget varies ($n = 2,000$ test). Performance peaks below the checkpoint’s configured maximum of 512 positions and degrades once the premise exceeds it.

Chars	Med. tok.	Acc.	F1	AUC-ROC
400	85	0.520	0.649	0.507
800 (main)	168	0.553	0.609	0.567
1600	329	0.598	0.611	0.629
3200	629	0.530	0.643	0.577

sources have a median length of 3,458 characters (mean 3,905, p90 6,825). The detector therefore never sees roughly three quarters of a typical document. We run two controlled follow-ups to separate the two explanations. Both keep the same NLI model and the same CPU-only budget, so only the input handling changes.

Does a larger premise help? We vary the source-truncation budget and rerun the detector on summarisation, keeping the protocol otherwise identical (Table 5). Performance is not flat. It climbs from 0.507 AUC-ROC at 400 characters to 0.629 at 1,600, then falls back to 0.577 at 3,200. The turning point is informative. The checkpoint is configured for at most 512 positions, and a 3,200-character premise already produces a median of 629 tokens. Because DeBERTa-v3 uses relative position attention, such inputs still run without error rather than being truncated, but they exceed the length the model is configured for, and its entailment judgments degrade. The budget used for our main results, 800 characters, sits well below that ceiling and leaves roughly six AUC-ROC points unclaimed. The truncation length is thus a real confound, though the ceiling it eventually runs into is a genuine property of the encoder rather than a free parameter.

Does removing truncation entirely help more?

A larger budget only postpones the problem, because no single pass can reliably exceed that length.

Table 6: Removing truncation via chunk aggregation on summarisation ($n = 2,000$ test). All rows use the same NLI model and the same instances, so only the input handling differs.

Method	Acc.	F1	AUC-ROC
Single-pass (800 chars)	0.553	0.609	0.567
Chunk agg. (min combiner)	0.588	0.648	0.633
Chunk agg. (mean combiner)	0.635	0.623	0.683

We therefore test a SummaC-style alternative (Laban et al., 2022) that avoids truncation altogether. The source is split into 800-character chunks and the candidate into sentences; every candidate sentence is scored against every chunk; entailment is maximised over chunks and then combined across candidate sentences, using either the mean (the SummaC-ZS default aggregation) or the minimum, which is more sensitive to a single unsupported sentence. Unlike SummaC-ZS, which scores entailment minus contradiction, we use the entailment probability alone. We deliberately reuse our own NLI model rather than the released SummaC checkpoint, so that the comparison isolates the aggregation strategy instead of confounding it with a different model. Table 6 reports the result on the full summarisation split against a single-pass baseline over the same instances; that baseline reproduces the 0.567 of Table 2 exactly. Aggregation with the mean combiner raises AUC-ROC to 0.683, a gain of 11.6 points, and improves accuracy and F1 as well, so it dominates single-pass scoring on every metric we report. It does so with no GPU and no larger model, though it needs about twenty times as many NLI calls. The two combiners trade off as expected: the minimum is the more sensitive detector and gives the best F1, while the mean gives the better ranking.

What this changes. Taken together, the two experiments substantially qualify the headline result. Summarisation is still clearly the hardest of the three tasks, and even the best configuration we find (0.683) remains far below QA (0.873) and dialogue (0.749). But the near-chance figures in Table 2 measure a naive single-pass configuration, not the capability of lightweight methods in general. Changes that need no additional hardware recover a large part of the gap: a larger premise budget, which lengthens each NLI input, and chunk aggregation, which needs roughly twenty times as many NLI calls. We therefore state the finding narrowly:

single-pass, truncation-bound lightweight detection fails on long-source summarisation, and practitioners working under CPU constraints should try aggregation, budgeting for its extra inference, before concluding that the task is out of reach.

7 Error Analysis and Discussion

NLI errors. On QA, false positives are short entity-only answers (“Queen Margrethe II”, “two”), where the brief candidate offers too little material to confirm entailment. False negatives are fluent fabrications (“Belk was founded in Birmingham, Alabama”) that read as locally plausible. The model is fooled more easily by confident falsehoods than by terse truths. On dialogue, false negatives involve plausible cross-references (e.g. a wrong film director) that fit the conversational flow but contradict the knowledge. On summarisation, false negatives differ from faithful summaries by a single altered fact that the truncated premise cannot reach.

Practical guidance. For both QA and dialogue, use the similarity-NLI ensemble as the default: it ranks first on F1 and AUC-ROC on both tasks. Where false positives are costly, standalone NLI gives the highest precision on QA; where recall matters most, standalone NLI is preferable on dialogue. For summarisation, none of the single-pass methods we test is usable; sentence-level chunk aggregation is the best option we found, at roughly twenty times the NLI inference, and claim decomposition is worth exploring.

8 Conclusion

We asked how far hallucination detection can progress without a GPU, proprietary APIs, or access to the generating model. Across four CPU-feasible detectors, a score-level ensemble, and three HaluEval tasks, the answer is strongly task-dependent. A similarity-NLI ensemble is the most consistent choice: it detects QA hallucinations well ($F1 = 0.792$, $AUC-ROC = 0.873$) and also ranks first on dialogue ($F1 = 0.694$, $AUC-ROC = 0.749$), where NLI is the strongest standalone method. Paired bootstrap comparisons confirm that the ensemble’s AUC-ROC lead is resolved on both tasks, while the ranking on summarisation is not. Every method performs near chance on summarisation ($AUC-ROC \leq 0.574$), but that figure describes a single-pass configuration rather than a ceiling: raising the premise budget recovers about six AUC-ROC points, and sentence-level

chunk aggregation recovers about twelve, the latter at roughly twenty times the NLI inference and without extra hardware. Summarisation remains the hardest of the three tasks, and the honest conclusion is narrower than the one we set out to draw. Truncation-bound single-pass scoring fails on long-source summarisation, and our results do not support treating lightweight detection as intrinsically unsuited to the task. We restrict attention to reproducible, public, CPU-only methods and report both successes and limits. This gives practitioners without specialised hardware a realistic foundation and clear guidance for choosing a method.

Limitations

We study only HaluEval, whose hallucinations are synthetic and may differ from naturally occurring ones; generalisation to hallucinations produced in the wild remains untested. Our conclusions rest on a single sampled split with one seed, so the bootstrap intervals we report capture test-set variability but not variation from resampling the corpus or reshuffling the validation split, and the gains in Section 6 are point estimates without intervals, because we did not retain per-instance scores for those runs. Detector orientation is fixed in advance rather than selected on validation, which is why BERTScore can fall below chance on summarisation. The chunk aggregation of Section 6 is a SummaC-style reimplementaion over our own NLI model rather than the released SummaC checkpoint, so it should be read as evidence about aggregation strategy rather than as a benchmark of that system; we also do not tune its chunk size, caps, or combiner, so the figures we report are a lower bound on what aggregation can achieve. Our comparison covers lexical, embedding, and entailment methods, but not other CPU-feasible families such as question-generation or claim-decomposition pipelines. Finally, the ensemble is a simple linear combination, and we leave learned stacking of all four detectors to future work.

Acknowledgments

This work received no external funding and was carried out on personal hardware. The authors declare no competing interests.

References

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, and

- Dario Amodei. 2020. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901.
- Tobias Falke, Leonardo F. R. Ribeiro, Prasetya Ajie Utama, Ido Dagan, and Iryna Gurevych. 2019. Ranking generated summaries by correctness: An interesting but challenging application for natural language inference. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2214–2220.
- Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, and Pascale Fung. 2023. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38.
- Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. 2022. SummaC: Re-visiting NLI-based models for inconsistency detection in summarization. *Transactions of the Association for Computational Linguistics*, 10:163–177.
- Moritz Laurer, Wouter Van Atteveldt, Andreu Casas, and Kasper Welbers. 2024. Less annotating, more classifying: Addressing the data scarcity issue of supervised machine learning with deep transfer learning and BERT-NLI. *Political Analysis*, 32(1):84–100.
- Junyi Li, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. 2023. HaluEval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6449–6464.
- Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81.
- Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522.
- Potsawee Manakul, Adian Liusie, and Mark J. F. Gales. 2023. SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9004–9017.
- Joshua Maynez, Shashi Narayan, Bernd Bohnet, and Ryan McDonald. 2020. On faithfulness and factuality in abstractive summarization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1906–1919.
- Sewon Min, Kalpesh Krishna, Xinxin Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FactScore: Fine-grained atomic evaluation of factual precision in long-form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100.
- Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, and Édouard Duchesnay. 2011. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992.
- James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: A large-scale dataset for fact extraction and verification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 809–819.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, and Guillaume Lample. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.
- Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating text generation with BERT. In *Proceedings of the 8th International Conference on Learning Representations*.

A Reproducibility Details

All experiments use fixed random seed 42 and run on CPU. The hardware is a laptop with an Apple M4 processor (10 cores) and 16 GB of RAM, running macOS 26.2 and Python 3.9.6; no GPU or accelerator is used at any stage, and we leave PyTorch threading at its default setting. We use three public model checkpoints: all-MiniLM-L6-v2 (semantic similarity), distilbert-base-uncased (BERTScore backbone), and MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli (NLI). For each task we draw 3,000 examples. We split them into a 1,000-instance validation set for threshold and ensemble-weight selection, and a 2,000-instance test set for reporting. We truncate the NLI premise to 800 source characters plus up to 200 trailing context characters and BERTScore references to 512 characters, and all-MiniLM-L6-v2 truncates its inputs at its default maximum of 256 tokens. ROUGE-L, semantic similarity, and

BERTScore use their raw scores as hallucination scores; NLI uses $1 - P(\text{entailment})$. We batch sentence embeddings at 32 and NLI inference at 16. Thresholds maximise the validation Youden index. The ensemble weight α maximises validation AUC-ROC over $\{0.1, \dots, 0.9\}$. We compute metrics with scikit-learn.

For the truncation ablation of Section 6 we vary only the source-character budget and leave the protocol otherwise unchanged, refitting the threshold on validation at each setting. For chunk aggregation we split the source into non-overlapping 800-character chunks, capped at 10 chunks, and the candidate into sentences on terminal punctuation, capped at 6; every candidate sentence is paired with every chunk, entailment is maximised over chunks and then combined across candidate sentences by mean or minimum. Bootstrap intervals use 2,000 replicates with seed 42.

B Bootstrap Confidence Intervals

Table 7 reports 95% bootstrap intervals for every method and task, computed by resampling the test set with replacement over 2,000 replicates (seed 42) and recomputing each metric on every replicate. Intervals are narrow, which reflects the 2,000-instance test splits.

Table 7: 95% bootstrap confidence intervals (2,000 replicates).

Task	Method	AUC-ROC	95% CI
QA	ROUGE-L	0.818	[0.797, 0.838]
	Sem. Sim.	0.736	[0.715, 0.757]
	BERTScore	0.821	[0.802, 0.838]
	NLI	0.795	[0.774, 0.816]
	Ensemble	0.873	[0.858, 0.888]
Dial.	ROUGE-L	0.612	[0.586, 0.638]
	Sem. Sim.	0.656	[0.632, 0.679]
	BERTScore	0.637	[0.614, 0.661]
	NLI	0.714	[0.692, 0.735]
	Ensemble	0.749	[0.729, 0.770]
Summ.	ROUGE-L	0.563	[0.538, 0.588]
	Sem. Sim.	0.528	[0.503, 0.553]
	BERTScore	0.468	[0.442, 0.493]
	NLI	0.567	[0.542, 0.592]
	Ensemble	0.574	[0.549, 0.598]

C Supplementary Figures

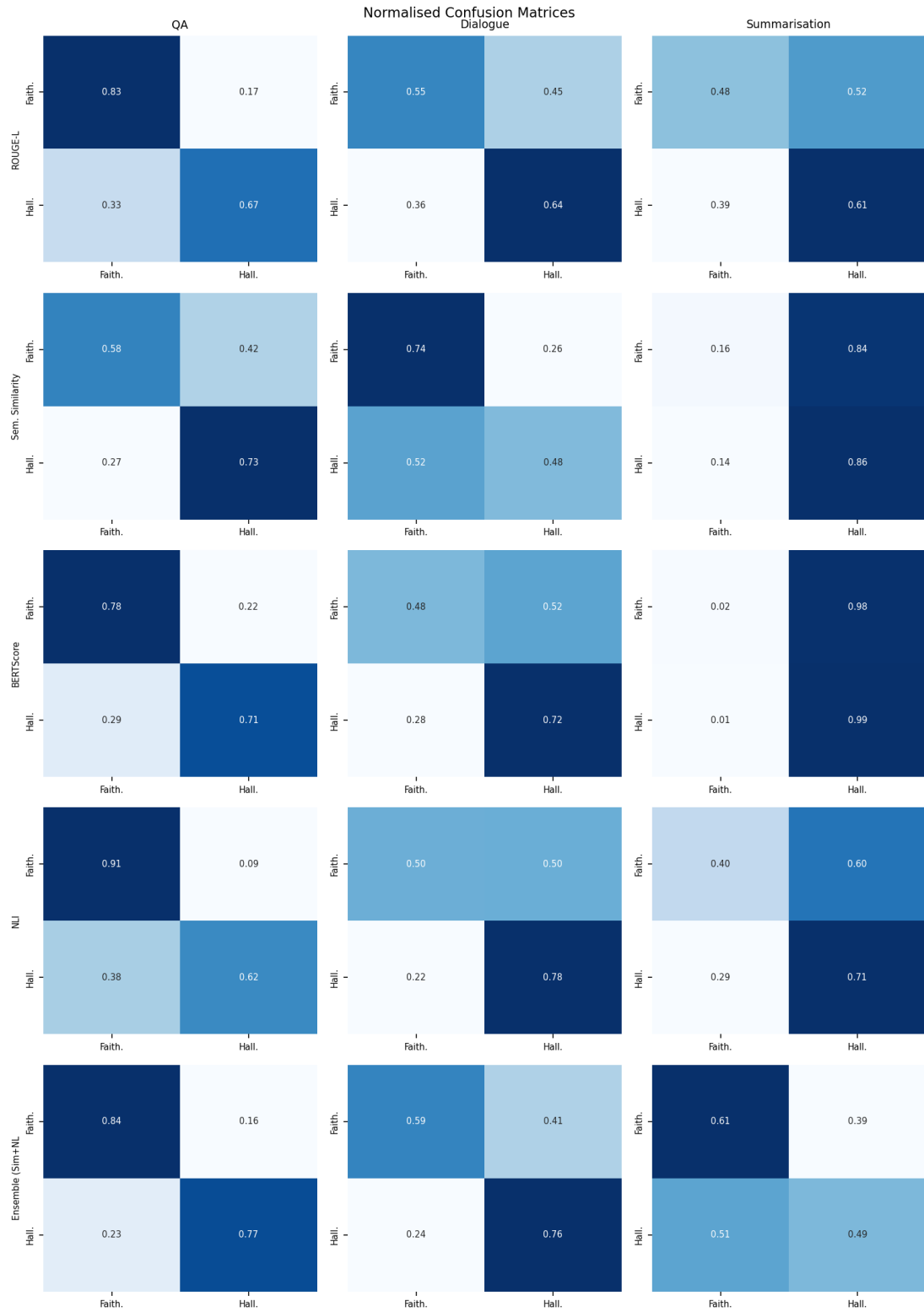


Figure 3: Confusion matrices for all methods and tasks at the calibrated operating point. The summarisation column shows the degenerate predict-almost-everything- hallucinated behaviour of the overlap methods.

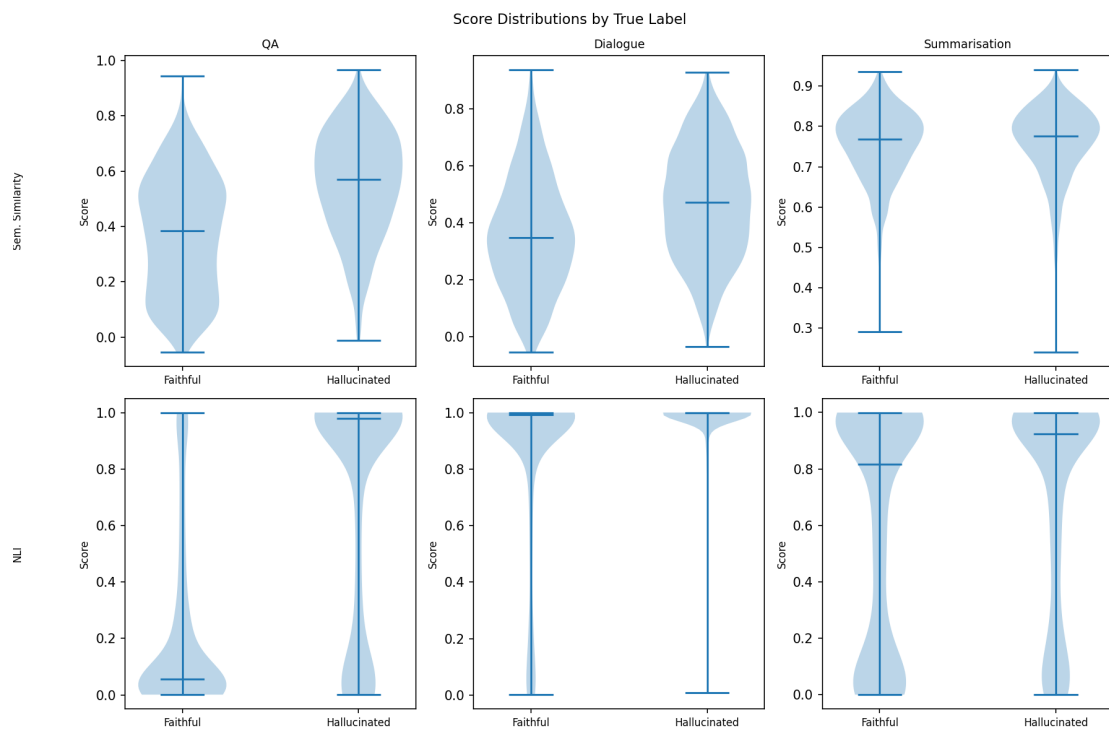


Figure 4: Distributions of hallucination scores for faithful versus hallucinated candidates. Clear separation on QA collapses to near-complete overlap on summarisation, explaining the near-chance AUC-ROC there.